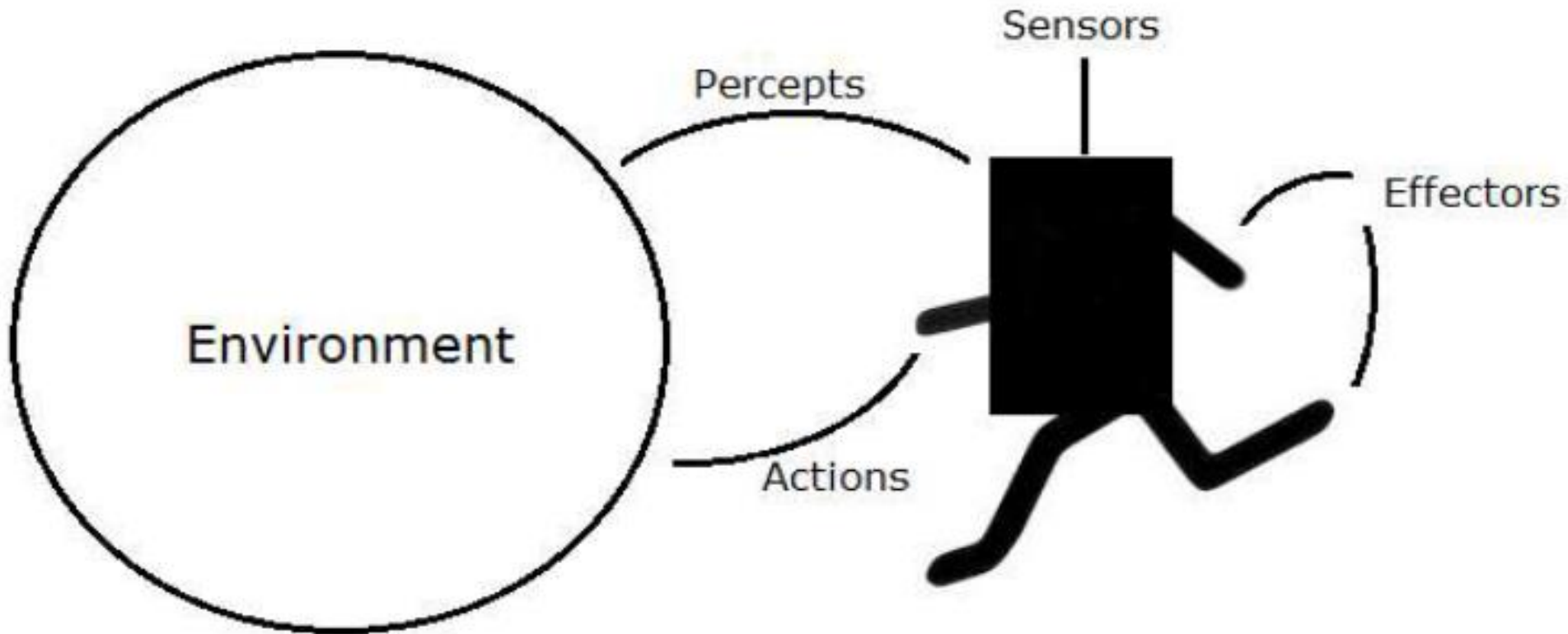


Chapter 2

INTELLIGENT AGENTS

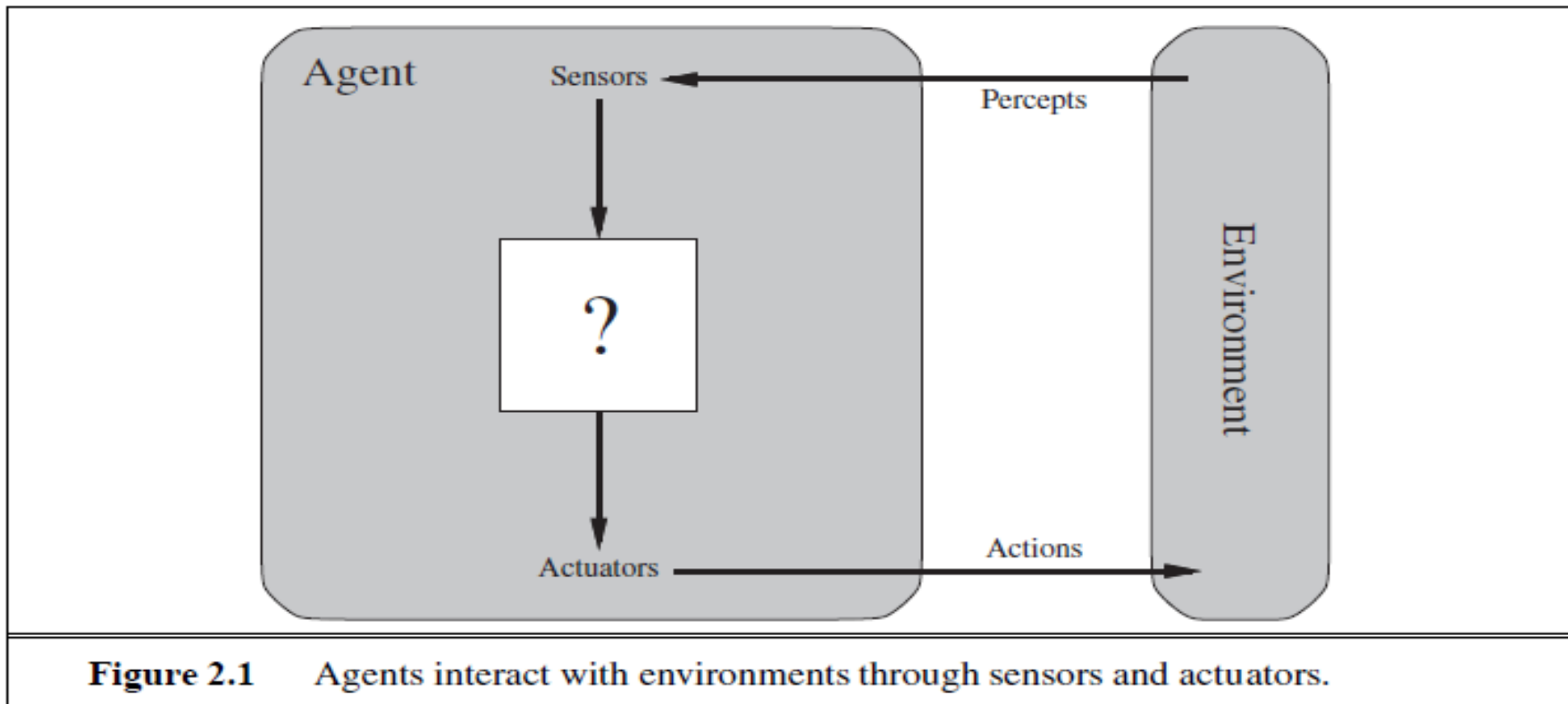
INTELLIGENT AGENTS

An **agent** is anything that can be viewed as perceiving its **environment** through **sensors** and **SENSOR** acting upon that environment through **actuators**.



INTELLIGENT AGENTS

An **agent** is anything that can be viewed as perceiving its **environment** through **sensors** and **SENSOR** acting upon that environment through **actuators**.



INTELLIGENT AGENTS

- **A human agent** has eyes, ears, and other organs for sensors and hands, legs, vocal tract, and so on for actuators.
- **A robotic agent** might have cameras and infrared range finders for sensors and various motors for actuators.
- A software agent receives keystrokes, file contents, and network packets as sensory inputs and acts on the environment by displaying on the screen, writing files, and sending network packets.

Agents Terminology

- **Performance Measure of Agent:** It is the criteria, which determines how successful an agent is.
- **Behavior of Agent:** It is the action that agent performs after any given sequence of percepts.
- **Percept:** It is agent's perceptual inputs at a given instance.
- **Percept Sequence:** It is the history of all that an agent has perceived till date.
- **Agent Function:** It is a map from the precept sequence to an action.

The agent program runs on the physical architecture to produce f

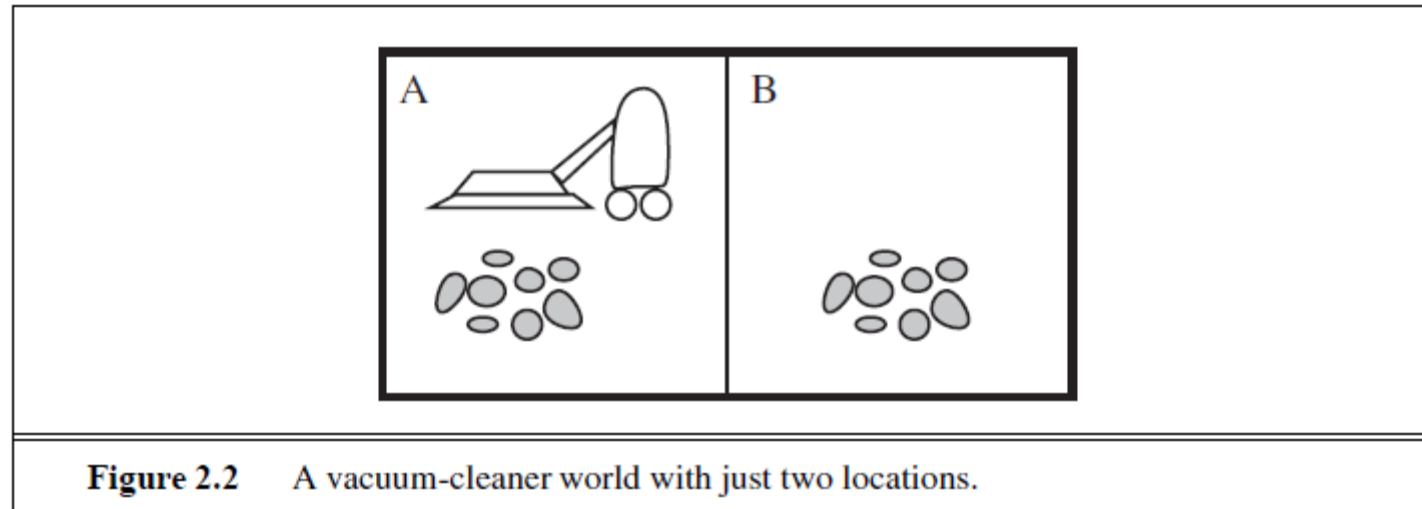
- agent = architecture + program

INTELLIGENT AGENTS

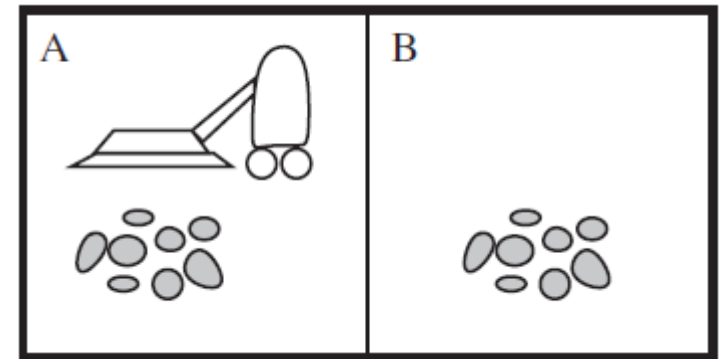
An agent's behavior is described by the agent function that maps any given percept sequence to an action.

$$[f: P^* \rightarrow A]$$

The agent function is an abstract mathematical description; the agent program is a concrete implementation, running within some physical system.



INTELLIGENT AGENTS



Percept sequence	Action
$[A, \textit{Clean}]$	<i>Right</i>
$[A, \textit{Dirty}]$	<i>Suck</i>
$[B, \textit{Clean}]$	<i>Left</i>
$[B, \textit{Dirty}]$	<i>Suck</i>
$[A, \textit{Clean}], [A, \textit{Clean}]$	<i>Right</i>
$[A, \textit{Clean}], [A, \textit{Dirty}]$	<i>Suck</i>
$\vdots$	$\vdots$
$[A, \textit{Clean}], [A, \textit{Clean}], [A, \textit{Clean}]$	<i>Right</i>
$[A, \textit{Clean}], [A, \textit{Clean}], [A, \textit{Dirty}]$	<i>Suck</i>
$\vdots$	$\vdots$

Figure 2.3 Partial tabulation of a simple agent function for the vacuum-cleaner world shown in Figure 2.2.

Rational agent:

For each possible percept sequence, a rational agent should select an action that is expected to maximize its performance measure, given the evidence provided by the percept sequence and whatever built-in knowledge the agent has.

Rational at any given time depends on four things:

- The performance measure that defines the criterion of success.
- The agent's prior knowledge of the environment.
- The actions that the agent can perform.
- The agent's percept sequence to date.

Rational agent

When an agent is plunked down in an environment, it generates a sequence of actions according to the percepts it receives.

This sequence of actions causes the environment to go through a sequence of states.

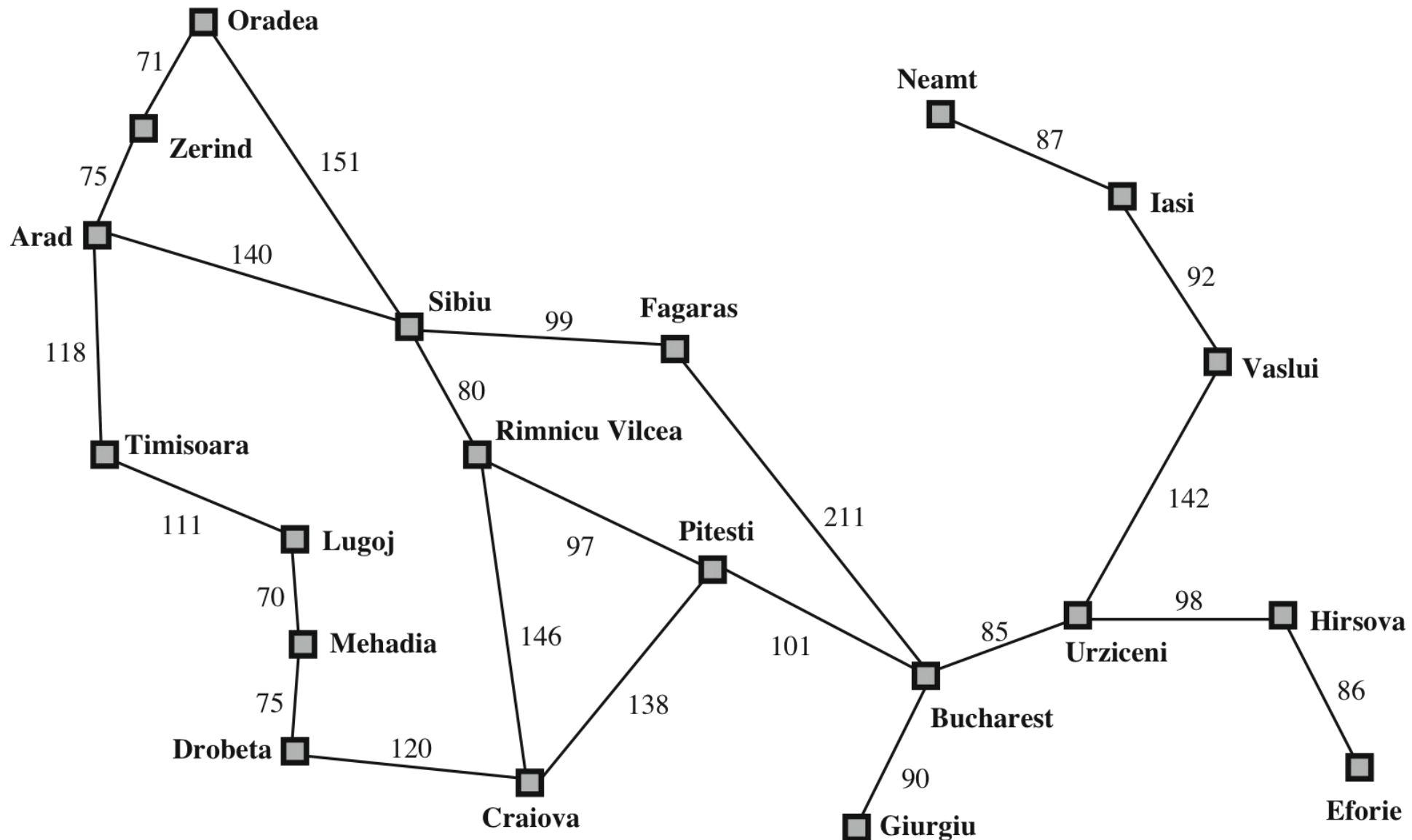
If the sequence is desirable, then the agent has performed well. This notion of desirability is captured by a performance measure that evaluates any given sequence of environment states.

Examples of Rational Choice

Consider the simplified map of Romania below. Numbers correspond to travel times. Consider the following choices.

1. To go from Rimnicu Vilcea to Craiova:
 - a. RV – Pitesti – Craiova, or
 - b. RV- Craiova?
2. To go from Oradea to Craiova:
 - a. Go via Arad-Sibiu-..., or
 - b. Go via Sibui, or
 - c. Go via Arad-Timisoara-....?

Examples of Rational Choice



Rationality

- Rational is different from omniscience
 - Percepts may not supply all relevant information
 - E.g., in card game, don't know cards of others.
- Rational is different from being perfect
 - Rationality maximizes expected outcome while perfection maximizes actual outcome.

PEAS

In designing an agent, the first step must always be to specify the task environment as fully as possible.

task environment means (the performance measure, the environment, and the agent's actuators and sensors).

we call them the **PEAS** (**P**erformance, **E**nvironment, **A**ctuators, **S**ensors).

Agent Type	Performance Measure	Environment	Actuators	Sensors
Taxi driver	Safe, fast, legal, comfortable trip, maximize profits	Roads, other traffic, pedestrians, customers	Steering, accelerator, brake, signal, horn, display	Cameras, sonar, speedometer, GPS, odometer, accelerometer, engine sensors, keyboard

Figure 2.4 PEAS description of the task environment for an automated taxi.

Task Environment

Agent Type	Performance Measure	Environment	Actuators	Sensors
Medical diagnosis system	Healthy patient, reduced costs	Patient, hospital, staff	Display of questions, tests, diagnoses, treatments, referrals	Keyboard entry of symptoms, findings, patient's answers
Satellite image analysis system	Correct image categorization	Downlink from orbiting satellite	Display of scene categorization	Color pixel arrays
Part-picking robot	Percentage of parts in correct bins	Conveyor belt with parts; bins	Jointed arm and hand	Camera, joint angle sensors
Refinery controller	Purity, yield, safety	Refinery, operators	Valves, pumps, heaters, displays	Temperature, pressure, chemical sensors
Interactive English tutor	Student's score on test	Set of students, testing agency	Display of exercises, suggestions, corrections	Keyboard entry

Figure 2.5 Examples of agent types and their PEAS descriptions.

Properties of Task Environments

(Environment types)

- **Fully observable** (vs. partially observable)
- **Deterministic** (vs. stochastic)
- **Episodic** (vs. sequential)
- **Static** (vs. dynamic)
- **Discrete** (vs. continuous)
- **Single agent** (vs. multiagent):

Properties of task environments

Fully observable vs. partially observable:

- If an agent's sensors give it access to the complete state of the environment at each point in time, then we say that the task environment is fully observable.
- A task environment is effectively fully observable if the sensors detect all aspects that are *relevant* to the choice of action; relevance, in turn, depends on the performance measure.
- An environment might be partially observable because of noisy and inaccurate sensors or because parts of the state are simply missing from the sensor data—for example, a vacuum agent with only a local dirt sensor cannot tell whether there is dirt in other squares, and an automated taxi cannot see what other drivers are thinking.
- If the agent has no sensors at all then the environment is unobservable.

Properties of task environments

Single agent vs. multiagent:

Single agent An agent operating by itself in an environment.

multiagent may be :

- **Competitive** (in chess, the opponent entity B is trying to maximize its performance measure, which, by the rules of chess, minimizes agent A's performance measure)
- **Cooperative** (In the taxi-driving environment, on the other hand, avoiding collisions maximizes the performance measure of COOPERATIVE all agents)

communication often emerges as a rational behavior in multiagent environments

Properties of task environments

Deterministic vs. stochastic.

If the next state of the environment is completely determined by the current state and the action executed by the agent, then we say the environment is deterministic; otherwise, it is stochastic.

In principle, an agent need not worry about uncertainty in a fully observable, deterministic environment.

Taxi driving is clearly stochastic in this sense, because one can never predict the behavior of traffic exactly.

The vacuum world as we described it is deterministic.

We say an environment is **uncertain** if it is not fully observable or not deterministic.

Properties of task environments

Episodic vs. sequential:

- In an episodic task environment, the agent's experience is divided into atomic episodes. In each episode the agent receives a percept and then performs a single action. Crucially, the next episode does not depend on the actions taken in previous episodes.
- In sequential environments, on the other hand, the current decision could affect all future decisions.
- Chess and taxi driving are sequential: in both cases, short-term actions can have long-term consequences.
- Episodic environments are much simpler than sequential environments because the agent does not need to think ahead.

Properties of task environments

Static vs. dynamic:

If the environment can change while an agent is deliberating, then the environment is dynamic for that agent.

Static environments are easy to deal with because the agent need not keep looking at the world while it is deciding on an action, nor need it worry about the passage of time.

Dynamic environments, on the other hand, are continuously asking the agent what it wants to do; if it hasn't decided yet, that counts as deciding to do nothing.

If the environment itself does not change with the passage of time but the agent's performance score does, then we say the environment is **semidynamic**.

Taxi driving is clearly dynamic. Crossword puzzles are static.
Chess, when played with a clock, is semidynamic.

Properties of task environments

Discrete vs. continuous:

The discrete/continuous distinction applies to the *state* of the environment, to the way *time* is handled, and to the *percepts* and *actions* of the agent.

For example, the chess environment has a finite number of distinct states (excluding the clock). Chess also has a discrete set of percepts and actions.

Taxi driving is a continuous-state and continuous-time problem: the speed and location of the taxi and of the other vehicles sweep through a range of continuous values and do so smoothly over time.

Properties of task environments

Known vs. unknown:

Strictly speaking, this distinction refers not to the environment itself but to the agent's (or designer's) state of knowledge about the “laws of physics” of the environment.

In a known environment, the outcomes (or outcome probabilities if the environment is stochastic) for all actions are given.

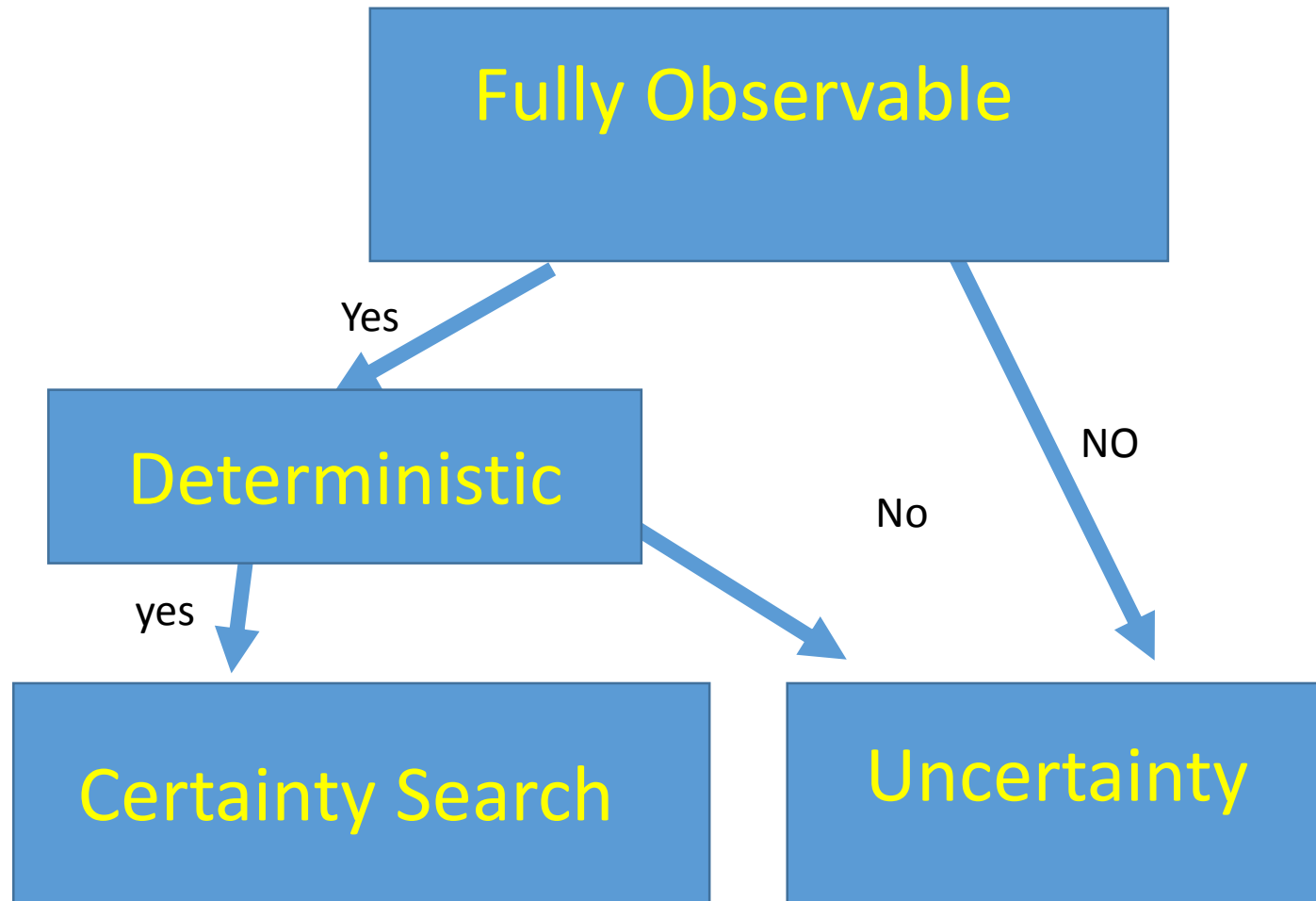
Obviously, if the environment is unknown, the agent will have to learn how it works in order to make good decisions. It is quite possible for a *known* environment to be *partially* observable—for example, in solitaire card games, I know the rules but am still unable to see the cards that have not yet been turned over. Conversely, an *unknown* environment can be *fully* observable—in a new video game, the screen may show the entire game state but I still don't know what the buttons do until I try them.

Properties of task environments

Task Environment	Observable	Agents	Deterministic	Episodic	Static	Discrete
Crossword puzzle	Fully	Single	Deterministic	Sequential	Static	Discrete
Chess with a clock	Fully	Multi	Deterministic	Sequential	Semi	Discrete
Poker	Partially	Multi	Stochastic	Sequential	Static	Discrete
Backgammon	Fully	Multi	Stochastic	Sequential	Static	Discrete
Taxi driving	Partially	Multi	Stochastic	Sequential	Dynamic	Continuous
Medical diagnosis	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Image analysis	Fully	Single	Deterministic	Episodic	Semi	Continuous
Part-picking robot	Partially	Single	Stochastic	Episodic	Dynamic	Continuous
Refinery controller	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Interactive English tutor	Partially	Multi	Stochastic	Sequential	Dynamic	Discrete

Figure 2.6 Examples of task environments and their characteristics.

Choice under (Un)certainty



THE STRUCTURE OF AGENTS

The job of AI is to design an **agent program** that implements the agent function the mapping from percepts to actions.

We assume this program will run on some sort of E computing device with physical sensors and actuators we call this the **architecture**:

$$\textit{agent} = \textit{architecture} + \textit{program} .$$

- the program we choose has to be one that is appropriate for the architecture.
- If the program is going to recommend actions like *Walk*, the architecture had better have legs. The architecture might be just an ordinary PC, or it might be a robotic car with several onboard computers, cameras, and other sensors.
- In general, the architecture makes the percepts from the sensors available to the program, runs the program, and feeds the program's action choices to the actuators as they are generated.

Agent programs

the agent program takes the current percept as input from the sensors and return an action to the actuators.

```
function TABLE-DRIVEN-AGENT(percept) returns an action
  persistent: percepts, a sequence, initially empty
               table, a table of actions, indexed by percept sequences, initially fully specified

  append percept to the end of percepts
  action  $\leftarrow$  LOOKUP(percepts, table)
  return action
```

Figure 2.7 The TABLE-DRIVEN-AGENT program is invoked for each new percept and returns an action each time. It retains the complete percept sequence in memory.

Agent programs

Consider the automated taxi: the visual input from a single camera comes in at the rate of roughly 27 megabytes per second (30 frames per second, 640×480 pixels with 24 bits of color information). This gives a lookup table with over $10^{250,000,000,000}$ entries for an hour's driving.

the lookup table for chess—a tiny, well-behaved fragment of the real world would have at least 10^{150} entries.

The daunting size of these tables means that:

- (a) no physical agent in this universe will have the space to store the table,
- (b) the designer would not have time to create the table,
- (c) no agent could ever learn all the right table entries from its experience, and
- (d) even if the environment is simple enough to yield a feasible table size, the designer still has no guidance about how to fill in the table entries.

Agent programs

The key challenge for AI is to find out how to write programs that, to the extent possible, produce rational behavior from a smallish program rather than from a vast table.

For example, the huge tables of square roots used by engineers and schoolchildren prior to the 1970s have now been replaced by a five-line program for Newton's method running on electronic calculators.

kinds of agent programs :

- Simple reflex agents;
- Model-based reflex agents;
- Goal-based agents; and
- Utility-based agents.

Agent programs

Simple reflex agents

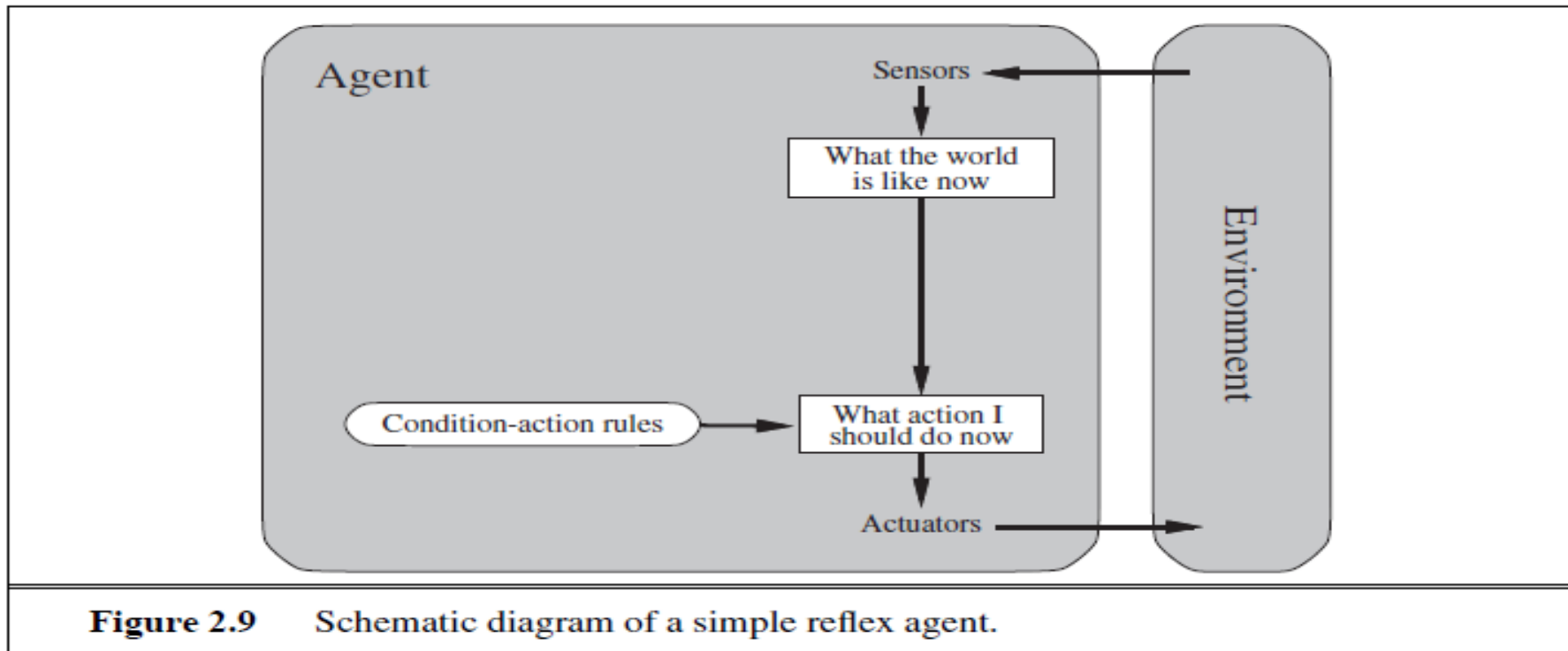
```
function SIMPLE-REFLEX-AGENT(percept) returns an action  
  persistent: rules, a set of condition–action rules  
  
  state ← INTERPRET-INPUT(percept)  
  rule ← RULE-MATCH(state, rules)  
  action ← rule.ACTION  
  return action
```

Figure 2.10 A simple reflex agent. It acts according to a rule whose condition matches the current state, as defined by the percept.

- The INTERPRET-INPUT function generates an abstracted description of the current state from the percept.
- The RULE-MATCH function returns the first rule in the set of rules that matches the given state description.
- Work only if the correct decision can be made on the basis of only the current percept that is, only if the environment is fully observable. Even a little bit of unobservability can cause serious trouble.

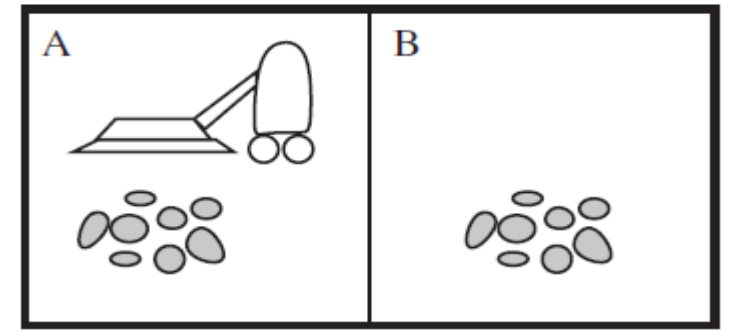
Agent programs

Simple reflex agents



Agent programs

Simple reflex agents



function REFLEX-VACUUM-AGENT(*[location,status]*) **returns** an action

if *status* = *Dirty* **then return** *Suck*

else if *location* = *A* **then return** *Right*

else if *location* = *B* **then return** *Left*

Figure 2.8 The agent program for a simple reflex agent in the two-state vacuum environment. This program implements the agent function tabulated in Figure 2.3.

These agents select actions on the basis of the *current* percept, ignoring the rest of the percept history

The vacuum agent is a simple reflex agent, because its decision is based only on the current location and on whether that location contains dirt.

Agent programs

Simple reflex agents

Imagine yourself as the driver of the automated taxi. If the car in front brakes and its brake lights come on, then you should notice this and initiate braking.

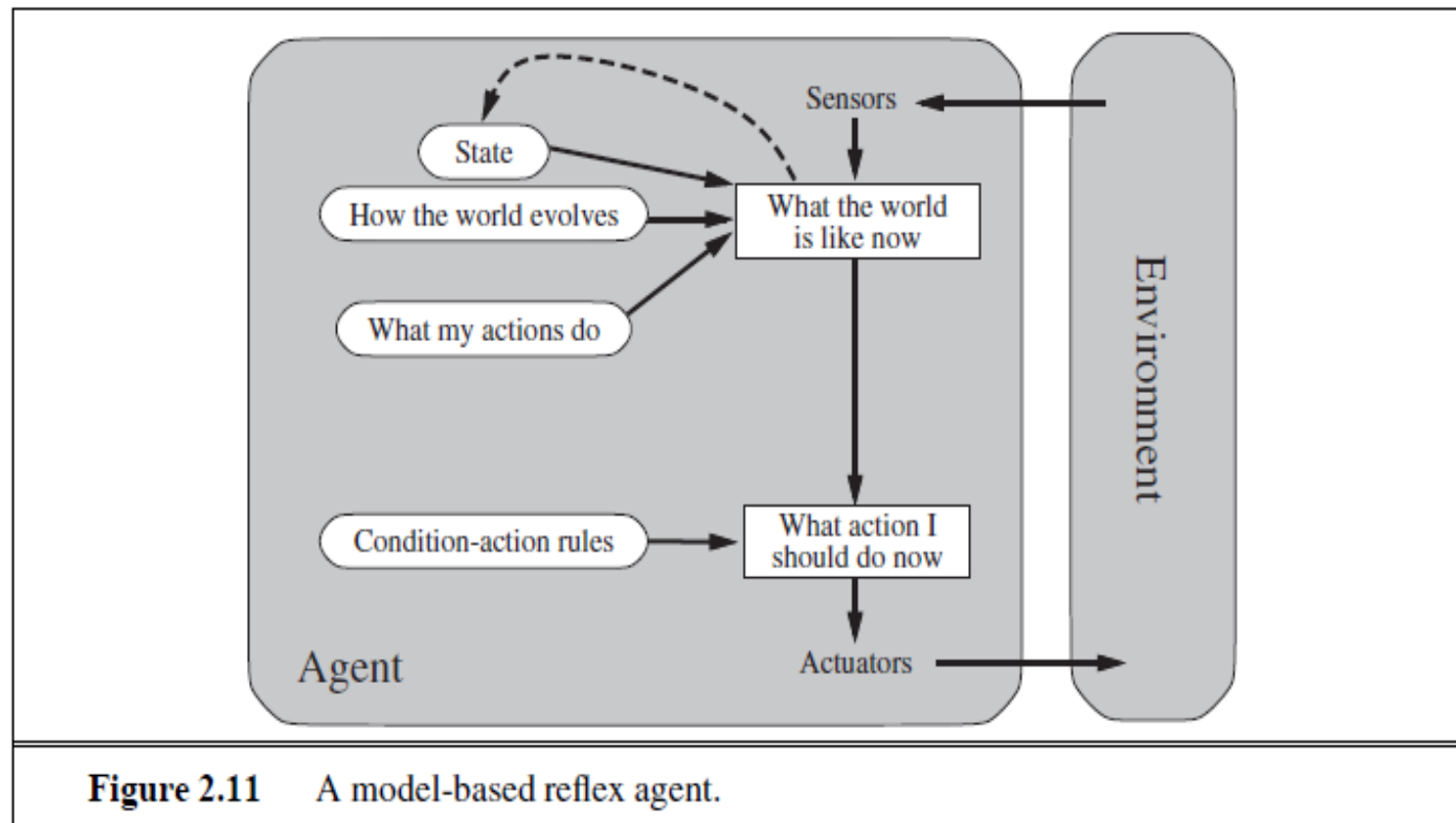
a condition–action rule

if *car-in-front-is-braking* **then** *initiate-braking*.

Agent programs

Model-based reflex agents

The most effective way to handle partial observability is for the agent to keep track of the part of the world it can't see now.



the current percept is combined with the old internal state to generate the updated description of the current state, based on the agent's model of how the world works.

Figure 2.11 A model-based reflex agent.

Agent programs

Model-based reflex agents

```
function MODEL-BASED-REFLEX-AGENT(percept) returns an action
  persistent: state, the agent's current conception of the world state
               model, a description of how the next state depends on current state and action
               rules, a set of condition–action rules
               action, the most recent action, initially none

  state ← UPDATE-STATE(state, action, percept, model)
  rule ← RULE-MATCH(state, rules)
  action ← rule.ACTION
  return action
```

Figure 2.12 A model-based reflex agent. It keeps track of the current state of the world, using an internal model. It then chooses an action in the same way as the reflex agent.

Agent programs

Model-based reflex agents

knowledge about “how the world works” whether implemented in simple Boolean circuits or in complete scientific theories is called a **model** of the world. An agent that uses such a model is called a **model-based agent**.

First, we need some information about how the world evolves independently of the agent for example, that an overtaking car generally will be closer behind than it was a moment ago.

Second, we need some information about how the agent’s own actions affect the world for example, that when the agent turns the steering wheel clockwise, the car turns to the right,

Agent programs

Goal-based agents

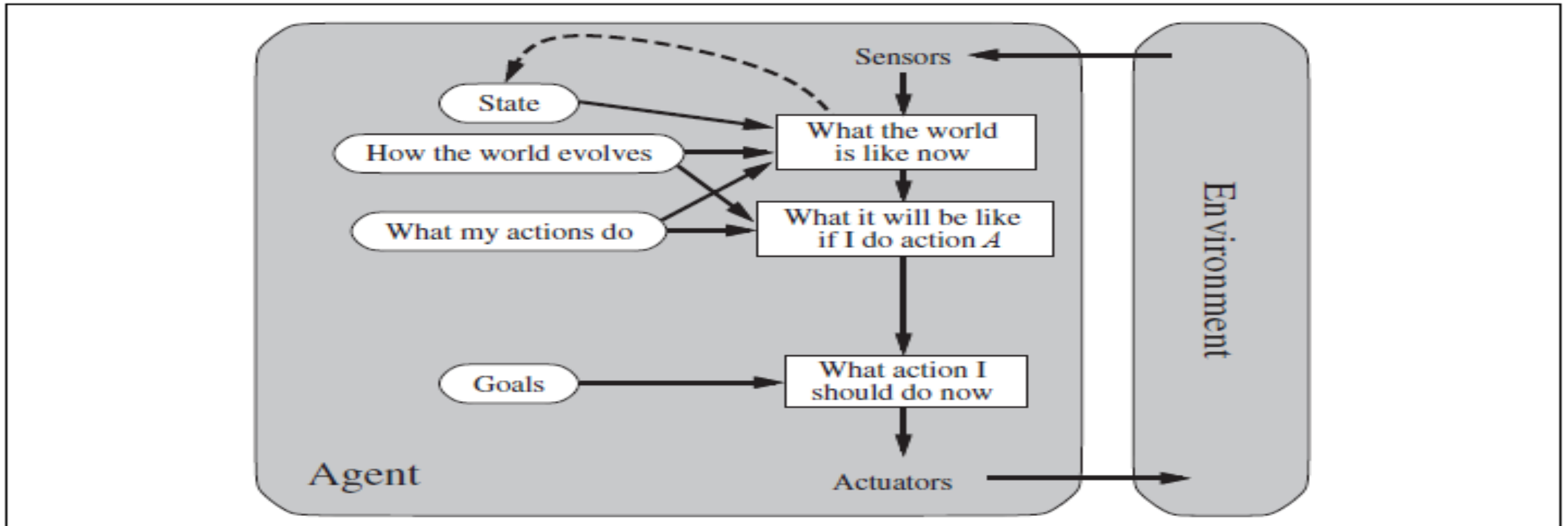


Figure 2.13 A model-based, goal-based agent. It keeps track of the world state as well as a set of goals it is trying to achieve, and chooses an action that will (eventually) lead to the achievement of its goals.

Agent programs

Goal-based agents

- Knowing something about the current state of the environment is not always enough to decide what to do.
- The agent needs some sort of **goal** information GOAL that describes situations that are desirable.
- The agent program can combine this with the model (the same information as was used in the model based reflex agent) to choose actions that achieve the goal.
- For example, at a road junction, the taxi can turn left, turn right, or go straight on. The correct decision depends on where the taxi is trying to get to.

Agent programs

Goal-based agents

- Sometimes it will be more tricky for example, when the agent has to consider long sequences of twists and turns in order to find a way to achieve the goal. **Search** and **planning** are the subfields of AI devoted to finding action sequences that achieve the agent's goals.
- Although the goal-based agent appears less efficient, it is more flexible because the knowledge that supports its decisions is represented explicitly and can be modified. If it starts to rain, the agent can update its knowledge of how effectively its brakes will operate; this will automatically cause all of the relevant behaviors to be altered to suit the new conditions.

Agent programs

Utility-based agents

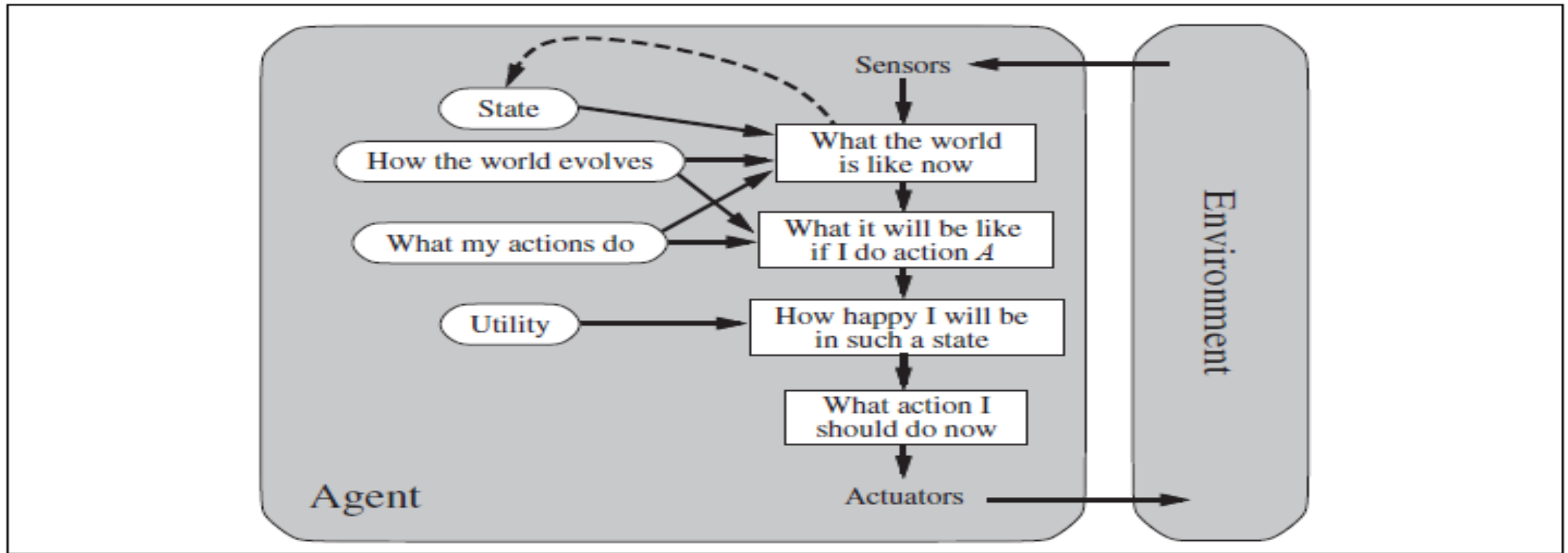


Figure 2.14 A model-based, utility-based agent. It uses a model of the world, along with a utility function that measures its preferences among states of the world. Then it chooses the action that leads to the best expected utility, where expected utility is computed by averaging over all possible outcome states, weighted by the probability of the outcome.

Agent programs

Utility-based agents

- Goals alone are not enough to generate high-quality behavior in most environments. For example, many action sequences will get the taxi to its destination (thereby achieving the goal) but some are quicker, safer, more reliable, or cheaper than others.
- A performance measure assigns a score to any given sequence of environment states, so it can easily distinguish between more and less desirable ways of getting to the taxi's destination.
- An agent's **utility function** is essentially an internalization of the performance measure. If the internal utility function and the external performance measure are in agreement, then an agent that chooses actions to maximize its utility will be rational according to the external performance measure.

Agent programs

Learning agents

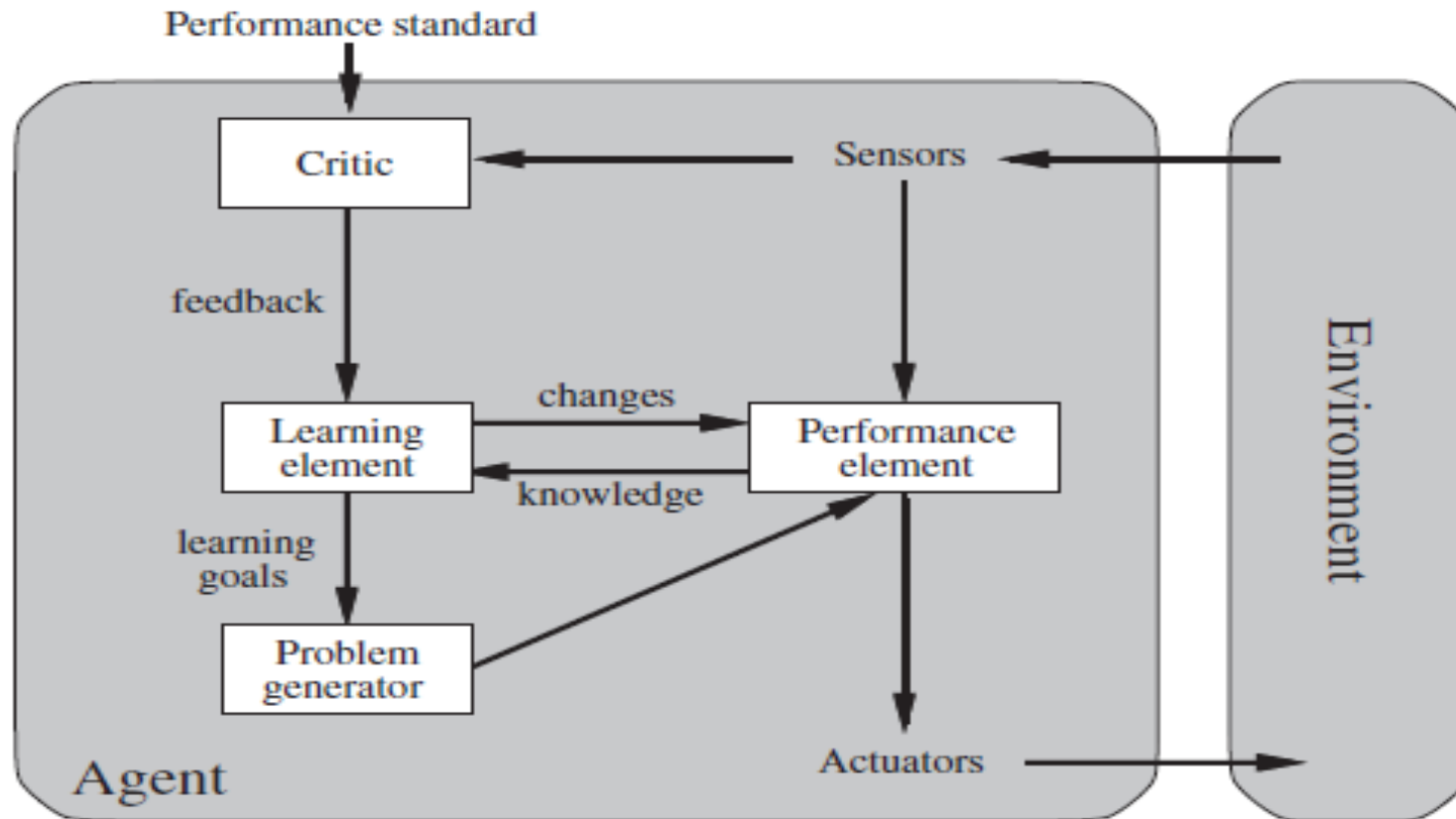


Figure 2.15 A general learning agent.

Agent programs

Learning agents

A learning agent can be divided into four conceptual components.

- The **learning element**, which is responsible for making improvements.
- The **performance element**, which is responsible for selecting external actions. The performance element is what we have previously considered to be the entire agent: it takes in percepts and decides on actions.
- The learning element uses feedback from the **critic** on how the agent is doing and determines how the performance element should be modified to do better in the future.
- The **critic** tells the learning element how well the agent is doing with respect to a fixed performance standard.
- the **problem generator**. It is responsible for suggesting actions that will lead to new and informative experiences.

Agent programs

Learning agents

To make the overall design more concrete, let us return to the automated taxi example. The performance element consists of whatever collection of knowledge and procedures the taxi has for selecting its driving actions. The taxi goes out on the road and drives, using this performance element. The critic observes the world and passes information along to the learning element. For example, after the taxi makes a quick left turn across three lanes of traffic, the critic observes the shocking language used by other drivers. From this experience, the learning element is able to formulate a rule saying this was a bad action, and the performance element is modified by installation of the new rule. The problem generator might identify certain areas of behavior in need of improvement and suggest experiments, such as trying out the brakes on different road surfaces under different conditions.

Learning agents(Taxi driver)

- Performance element
 - How it currently drives
- Taxi driver Makes quick left turn across 3 lanes
 - Critics observe shocking language by passenger and other drivers and informs bad action
 - Learning element tries to modify performance elements for future
 - Problem generator suggests experiment out something called Brakes on different Road conditions
- Exploration vs. Exploitation
 - Learning experience can be costly in the short run
 - shocking language from other drivers
 - Less tip
 - Fewer passengers